Binary search runs twice. It takes the first employee at or after the low time and the next search takes the first employee after the high time. After finding the lowest and highest, a for loop copies all the elements between the highest and lowest. Because the array already is sorted this is a very efficient search method for large amounts of data since it halves the array each iteration.
The note said the murderer entered and exited between 22:00 and 23:00 on 9 August 2234. So the search was between these two times. The four found were Jasper Wellington, Alfred Pugsley, Nem Wellington and Helena Blackwood. They all entered at 22:13:17 but by checking their exit times I found that Alfred Pugsley also exited during that time window which means he is the suspect.

Binary search tree was built from week two's unsorted array. Each node stores one time parsed to DateTime, a reference for one right and one left child and a list of all employees with the same time. That gives 525 nodes for the 800 employees. The insert checks the next time and adds it to the left or right child unless it's the exact same time in which case it gets added to the same time list. The actual search is efficient because it can skip all the nodes that are below or above the time window by not going down those children.
The search took about 4.7 microseconds for the binary search and 0.56 microseconds for the tree search over 10,000 runs to get a better estimate of how long it takes on average. Before I ran the searches I expected them to be pretty much the same speed since both halve the array of employees until they find the right one. But what I failed to account for was how I made the binary search parse the string to DateTime every comparison. The tree does it once before the search which most of the speed comes from. A better benchmark would have been to save the parsed data for both so it would become more fair.

A challenge I encountered was during the quicksort, while copying the employees into a new array to sort them I called my quicksort inside the copy loop and since everything hadn't been copied yet many of the slots were null and so the pivot could become null so the program crashed each time I started the search. It was an easy fix by just moving the function call out of the loop, but it took some time to find that it was inside one too many brackets.

Another challenge was that the tree search found all the right employees for each search but they always were unsorted. This was because whenever I found an employee that matched the range I added it to the results. I failed to realize that all left children would need to be added before the node holding them. I fixed it by adding the left child then the node then the right child.

While benchmarking I also had a challenge with the result. The tree benchmark said 0.000142 seconds, and binary search said 0.0795 seconds. So it looked way off, but I forgot to divide one of them by 10,000 to get the average. There was still a difference of about 18 times after this fix. That was because I printed a Debug.Log inside each iteration of the 10,000 loop for the tree search which took much longer than the search itself. After figuring this out I commented out the Debug.Log and ran again which produced numbers that made more sense.